



Experiment 14: Queue Sort

Aim

To write a program to **sort elements using Queue (Queue Sort)**.

Theory

Queue is a **linear data structure** that follows **FIFO (First In First Out)** principle.

In Queue Sort, elements are sorted by repeatedly finding the **minimum element** from the queue and placing it in sorted order.

This method is similar to **Selection Sort**, but uses queue operations like **enqueue** and **dequeue**.

Algorithm

QueueSort(Q)

```
1. Let n = number of elements in queue Q
2. for i = 0 to n-1 do
3.     min = ∞
4.     // Step 1: Find minimum element
5.     for j = 0 to n-i-1 do
6.         x = dequeue(Q)
7.         if x < min then
8.             min = x
9.         enqueue(Q, x)
10.
11.    // Step 2: Remove one occurrence of min
12.    flag = 0
13.    for j = 0 to n-i-1 do
14.        x = dequeue(Q)
15.        if x == min and flag == 0 then
16.            store min in output array
17.            flag = 1
18.        else
19.            enqueue(Q, x)
20.    end for
21. Print sorted elements
```



Program (C Language)

```
#include <stdio.h>
```

```
int main() {
    int q[100], sorted[100];
```

```

int n, i, j, min, pos, temp;

// Input
printf("Enter number of elements: ");
scanf("%d", &n);

printf("Enter elements: ");
for(i = 0; i < n; i++) {
    scanf("%d", &q[i]);
}

// Queue Sort (Simple logic)
for(i = 0; i < n; i++) {
    min = q[0];
    pos = 0;

    // Find minimum element
    for(j = 1; j < n - i; j++) {
        if(q[j] < min) {
            min = q[j];
            pos = j;
        }
    }

    // Store minimum in sorted array
    sorted[i] = min;

    // Shift remaining elements (simulate dequeue)
    for(j = pos; j < n - i - 1; j++) {
        q[j] = q[j + 1];
    }
}

// Output
printf("Sorted elements: ");
for(i = 0; i < n; i++) {
    printf("%d ", sorted[i]);
}

return 0;
}

```

Sample Input

Queue Elements:

4, 2, 7, 1, 3

Sample Output

Sorted Elements:

1 2 3 4 7

Result

The given elements are successfully sorted using **Queue Sort technique**.

Conclusion

Queue Sort is easy to implement using basic queue operations but has **$O(n^2)$** time complexity, so it is not suitable for large datasets.